

Design Patterns

Singleton Pattern in C++

- Ensures **only one instance** of a class exists throughout the program lifetime.
 - All users share the same instance via a **static method** exposed in the class.
-

Principles

1. **Private constructor & destructor**
 - Prevent direct creation or destruction of the object from outside.
 2. **Deleted copy constructor & assignment operator**
 - Prevent copying or assignment (no duplicates).
 3. **Static method** to return the instance.
 4. **Global point of access** to the single instance.
-

Initialization Types

1. Lazy Initialization

- Instance is created **only when first requested**.
- Saves resources when the instance is not used initially.
- Delays the cost of construction until needed.
- Preferred in most cases for efficiency.
- **C++11+**: Static local variable initialization is **thread-safe by default**.

- **Before C++11:** Required **double-checked locking** for thread safety.

2. Eager Initialization

- Instance is created **at program startup** (before it's used).
- Simpler but wastes resources if unused.
- No thread-safety concerns because initialization happens before `main()`.

```
// lazy initialization
#include <iostream>
using namespace std;

class Logger {
    Logger() = default;
    ~Logger() = default;
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;
public:
    static Logger& instance() {
        static Logger inst;
        return inst;
    }

    void log(const string& msg) const {
        cout<<"Log : "<<msg<<endl;
    }
};

int main() {
    Logger::instance().log("Start logging");
    Logger::instance().log("End Logging");

    return 0;
}
```

```

// eager initialization
#include <iostream>
using namespace std;

class Logger {
    Logger() = default;
    ~Logger() = default;
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;
public:
    static Logger inst;

    static Logger& instance() {
        return inst;
    }

    void log(const string& msg) const {
        cout<<"Log : "<<msg<<endl;
    }
};

Logger Logger::inst;

int main() {
    Logger::instance().log("Start logging");
    Logger::instance().log("End Logging");

    return 0;
}

```

Factory Method Design Pattern

The **Factory Method Pattern** is a creational design pattern that defines an interface for creating an object, but lets subclasses decide which class to instantiate.

Key Points

- **Purpose:** Encapsulates object creation logic.
- **Avoids:** Tight coupling between the client and concrete classes.
- Follows **Open/Closed Principle** (add new products without modifying existing code).

```
#include <iostream>

using namespace std;

// product interface
class Document {
public:
    virtual void open() = 0;
    virtual ~Document() = default;
};

// concrete Document
class WordDocument : public Document {
public:
    void open() {
        cout<<"Opening word document"<<endl;
    }
};

class PdfDocument : public Document {
public:
    void open() {
        cout<<"Opening pdf document"<<endl;
    }
};
```

```

class DocumentCreator {
public:
    virtual Document* createDocument() = 0;

    void openDocument() {
        auto doc = createDocument();
        doc->open();
    }

    virtual ~DocumentCreator() = default;
};

class WordDocumentCreator : public DocumentCreator {
public:
    Document* createDocument() {
        return new WordDocument();
    }
};

class PdfDocumentCreator : public DocumentCreator {
public:
    Document* createDocument() {
        return new PdfDocument();
    }
};

int main() {
    DocumentCreator* wordCreator = new WordDocumentCreator();
    wordCreator->openDocument();

    DocumentCreator* pdfCreator = new PdfDocumentCreator();
    pdfCreator->openDocument();

    return 0;
}

```

Abstract Factory Design Pattern

The **Abstract Factory Pattern** is a creational pattern that provides an **interface to create families of related or dependent objects** without specifying their concrete classes.

```
#include <iostream>

using namespace std;

// abstract product A
class Chair {
public:
    virtual void sitOn() = 0;
    virtual ~Chair() = default;
};

// abstract product B
class Sofa {
public:
    virtual void lieOn() = 0;
    virtual ~Sofa() = default;
};

// concrete Product Modern
class ModernChair : public Chair {
public:
    void sitOn() {
        cout<<"Sit on modern chair"<<endl;
    }
};

class ModernSofa : public Sofa {
public:
    void lieOn() {
        cout<<"Lie on modern sofa"<<endl;
    }
};

// concrete Product Victorian
```

```

class VictorianChair : public Chair {
public:
    void sitOn() {
        cout<<"Sit on victorian chair"<<endl;
    }
};

class VictorianSofa : public Sofa {
public:
    void lieOn() {
        cout<<"Lie on victorian sofa"<<endl;
    }
};

// Abstract factory
class FurnitureFactory {
public:
    virtual Chair* createChair() = 0;
    virtual Sofa* createSofa() = 0;
    virtual ~FurnitureFactory() = default;
};

// Concrete Factories
class ModernFurnitureFactory : public FurnitureFactory {
public:
    Chair* createChair() {
        return new ModernChair();
    }

    Sofa* createSofa() {
        return new ModernSofa();
    }
};

class VictorianFurnitureFactory : public FurnitureFactory {
public:
    Chair* createChair() {
        return new VictorianChair();
    }
};

```

```

    Sofa* createSofa() {
        return new VictorianSofa();
    }
};

// client context
void furnishRoom(FurnitureFactory& factory) {
    auto chair = factory.createChair();
    auto sofa = factory.createSofa();

    chair->sitOn();
    sofa->lieOn();
}

int main() {
    ModernFurnitureFactory modernFactory;
    VictorianFurnitureFactory victorianFactory;

    cout<<"Modern -"<<endl;
    furnishRoom(modernFactory);

    cout<<"Victorian -"<<endl;
    furnishRoom(victorianFactory);

    return 0;
}

```

Builder Design Pattern

The Builder Pattern is a creational design pattern that lets you **construct complex objects step-by-step**, separating the construction process from the object's representation.

Purpose: Build complex objects in a controlled sequence.

Highly used when using layered architecture.


```

#include <iostream>

using namespace std;

class Car {
    int seats;
    string engine;
    bool hasGps;

    Car(int seats, string engine, bool hasGps) :
        seats(seats), engine(engine), hasGps(hasGps) {}

public:
    int getSeats() { return seats; }
    string getEngine() { return engine; }
    bool getHasGps() { return hasGps; }

    friend class Builder;

    class Builder {
        int seats;
        string engine;
        bool hasGps;

    public:
        Builder() : seats(0), engine("unknown"), hasGps(false) {}

        Builder& setSeats(int seats) {
            this->seats = seats;
            return *this;
        }

        Builder& setEngine(string engine) {
            this->engine = engine;
            return *this;
        }

        Builder& setGps(bool hasGps) {
            this->hasGps = hasGps;
            return *this;
        }
    };
};

```

```

    }

    Car* build() {
        return new Car(seats, engine, hasGps);
    }
};

int main() {
    Car* car = Car::Builder().setSeats(4)
                             .setEngine("sports")
                             .setGps(false)
                             .build();
}

```

Prototype Design Pattern

The **Prototype Pattern** is a creational design pattern that creates new objects by **cloning** existing objects (called *prototypes*) instead of creating them from scratch.

Purpose: Reduces cost of creating complex objects.

```

#include <iostream>
#include <string>

using namespace std;

```

```

class Resume {
public:
    virtual Resume* clone() = 0;
    virtual void show() = 0;
    virtual ~Resume() {}
};

// concrete prototype
class DeveloperResume : public Resume {
    string name;
    string skills;
public:
    DeveloperResume(string name, string skills) : name(name), skills(skills) {}

    Resume* clone() {
        return new DeveloperResume(*this);
    }

    void setName(string newName) {
        name = newName;
    }

    void show() {
        cout<<"Developer Name -"<<name<<endl;
        cout<<"Developer Skills -"<<skills<<endl;
    }
};

int main() {
    Resume* original = new DeveloperResume("Alice", "C++");
    original->show();

    Resume* copy = original->clone();
    ((DeveloperResume*) copy)->setName("Bob");

    copy->show();

    delete original;
    delete copy;
}

```

Adapter Design Pattern

The **Adapter Pattern** is a **structural design pattern** that allows **incompatible interfaces** to work together by converting one interface into another expected by the client.

When you have existing code (legacy or third-party) with a different interface than what your system expects.

```
#include <iostream>

using namespace std;

// Target Interface
class PaymentProcessor {
public:
    virtual void pay(double amount) = 0;
    virtual ~PaymentProcessor() {}
};

class PayPalAPI {
public:
    void sendPayment(double amount) {
        cout<<"Paid "<<amount<<" using paypal"<<endl;
    }
};

class PayPalAdapter : public PaymentProcessor {
    PayPalAPI* paypal;
public:
    PayPalAdapter(PayPalAPI* p) : paypal(p) {}
    void pay(double amount) {
        paypal->sendPayment(amount);
    }
};

int main() {
    PayPalAPI paypalApi;
    PaymentProcessor* paypal = new PayPalAdapter(&paypalApi);
```

```
paypal->pay(50.0);

delete paypal;
return 0;
}
```

Bridge Pattern

The **Bridge Design Pattern** is a **structural pattern** that decouples an abstraction from its implementation so the two can vary independently.

Purpose: When you have multiple dimensions of variation

Example:

1. We want to control different devices (TV, Radio) using different types of remotes (Basic, Advanced).
2. Another example
 - Different payment gateways: PayPal, Stripe, Razorpay (implementation changes).
 - Different payment methods: One-time payment, Subscription, EMI (abstraction changes).
 - If you directly mix them then you have to take cross product, bridge pattern overcomes this part of code

```
#include <iostream>
#include <string>

using namespace std;

// implementor
class PaymentGateway {
Public:
    virtual void processPayment(double amount) = 0;
```

```

        virtual ~PaymentGateway() {}
    };

    // concrete implementor
    class PayPalGateway : public PaymentGateway {
    Public:
        void processPayment(double amount) {
            cout<<"Processing "<<amount<<" via PayPal"<<endl;
        }
    };

    // Abstraction
    class PaymentMethod {
    Protected:
        PaymentGateway* gateway;
    Public:
        PaymentMethod(PaymentGateway* g) : gateway(g) {}
        virtual void makePayment(double amount) = 0;
        virtual ~PaymentMethod() {}
    };

    // refined abstraction
    class OneTimePayment : public PaymentMethod {
    Public:
        OneTimePayment(PaymentGateway* g) : PaymentMethod(g) {}
        void makePayment(double amount) {
            cout<<"Init one-time payment"<<endl;
            gateway->processPayment(amount);
        }
    };

    class SubscriptionPayment : public PaymentMethod {
    Public:
        SubscriptionPayment(PaymentGateway* g) : PaymentMethod(g) {}
        void makePayment(double amount) {
            cout<<"Init subscription payment"<<endl;
            gateway->processPayment(amount);
        }
    };

```

```

int main() {
    PayPalGateway payPal;
    OneTimePayment otp(&payPal);
    otp.makePayment(100);

    SubscriptionPayment sub(&payPal);
    sub.makePayment(20);
    return 0;
}

```

Composite Design Pattern

Composite Pattern allows you to treat a group of objects (composite) and a single object (leaf) in the same way using a common interface.

It is mainly used to represent tree-like structures (e.g., file systems, UI elements, menus).

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

// Component
class FileSystemItem {
public:
    virtual void show(int indent = 0) = 0;
    virtual ~FileSystemItem() {}
};

```

```

// Leaf
class File : public FileSystemItem {
    string name;
public:
    File(string n) : name(n) {}
    void show(int indent = 0) override {
        cout << string(indent, '-') << "File: " << name << endl;
    }
};

// Composite
class Folder : public FileSystemItem {
    string name;
    vector<FileSystemItem*> items;
public:
    Folder(string n) : name(n) {}
    void add(FileSystemItem* item) { items.push_back(item); }

    void show(int indent = 0) override {
        cout << string(indent, '-') << "Folder: " << name << endl;
        for (auto item : items) item->show(indent + 2);
    }
};

// Client
int main() {
    Folder* root = new Folder("Root");
    root->add(new File("file1.txt"));
    root->add(new File("file2.txt"));

    Folder* subFolder = new Folder("Documents");
    subFolder->add(new File("doc1.pdf"));
    subFolder->add(new File("doc2.docx"));

    root->add(subFolder);

    root->show();
    return 0;
}

```


Decorator Design Pattern

The **Decorator Pattern** is a **structural pattern** that lets you add new behavior or responsibilities to objects **dynamically** without modifying their original class.

It wraps the original object inside a new class (the decorator) that adds extra functionality.

```
#include <iostream>
#include <string>
using namespace std;

// Component
class Text {
public:
    virtual string getContent() = 0;
    virtual ~Text() {}
};

// Concrete Component
class PlainText : public Text {
    string content;
public:
    PlainText(const string& c) : content(c) {}
    string getContent() override { return content; }
};

// Decorator (abstract)
class TextDecorator : public Text {
protected:
    Text* wrapped;
public:
    TextDecorator(Text* t) : wrapped(t) {}
    string getContent() override { return wrapped->getContent(); }
};

// Concrete Decorator - Bold
class BoldDecorator : public TextDecorator {
```

```

public:
    BoldDecorator(Text* t) : TextDecorator(t) {}
    string getContent() override {
        return "<b>" + TextDecorator::getContent() + "</b>";
    }
};

// Concrete Decorator - Italic
class ItalicDecorator : public TextDecorator {
public:
    ItalicDecorator(Text* t) : TextDecorator(t) {}
    string getContent() override {
        return "<i>" + TextDecorator::getContent() + "</i>";
    }
};

// Client
int main() {
    Text* plain = new PlainText("Hello World");

    Text* bold = new BoldDecorator(plain);
    Text* italic = new ItalicDecorator(plain);
    Text* boldItalic = new BoldDecorator(new ItalicDecorator(plain));

    cout << "Plain: " << plain->getContent() << endl;
    cout << "Bold: " << bold->getContent() << endl;
    cout << "Italic: " << italic->getContent() << endl;
    cout << "Bold + Italic: " << boldItalic->getContent() << endl;

    // cleanup
    delete boldItalic;
    delete italic;
    delete bold;
    delete plain;

    return 0;
}

```

Facade Design Pattern

The **Facade Pattern** is a **structural pattern** that provides a simplified interface to a larger, more complex subsystem.

Instead of exposing all the details of multiple classes, you create a Facade class that offers easy-to-use methods for the client.

```
#include <iostream>
using namespace std;

// Subsystems
class FlightBooking {
public:
    void bookFlight() { cout << "Flight booked\n"; }
};

class HotelBooking {
public:
    void bookHotel() { cout << "Hotel booked\n"; }
};

class CabBooking {
public:
    void bookCab() { cout << "Cab booked\n"; }
};

// Facade
class TravelAgency {
    FlightBooking flight;
    HotelBooking hotel;
    CabBooking cab;
public:
    void bookHolidayPackage() {
        cout << "\nBooking Holiday Package...\n";
        flight.bookFlight();
        hotel.bookHotel();
        cab.bookCab();
    }
};
```

```
        cout << "Holiday package booked successfully!\n";
    }
};

// Client
int main() {
    TravelAgency agency;
    agency.bookHolidayPackage(); // simple call for client
    return 0;
}
```

Flyweight Design Pattern

Flyweight is a structural design pattern that helps to minimize memory usage by sharing common, reusable objects instead of creating new ones every time.

It separates **intrinsic state** (shared, invariant data) from **extrinsic state** (context-specific data passed at runtime).

- When you need to create a **huge number of objects** (e.g., characters in a text editor, trees in a forest, particles in a game).
- When many objects share **common properties** that can be **reused**.

```

#include <iostream>
#include <map>
#include <string>
using namespace std;

// Flyweight
class CharacterStyle {
    string font;
    int size;
    string color;
public:
    CharacterStyle(string font, int size, string color)
        : font(font), size(size), color(color) {}

    void apply(int x, int y, char c) {
        cout << "Drawing '" << c
            << "' with [" << font << ", " << size << ", " << color
            << "]" at (" << x << ", " << y << ")\\n";
    }
};

// Flyweight Factory
class StyleFactory {
    map<string, CharacterStyle*> styles;
public:
    CharacterStyle* getStyle(string font, int size, string color) {
        string key = font + "_" + to_string(size) + "_" + color;
        if (styles.find(key) == styles.end()) {
            styles[key] = new CharacterStyle(font, size, color);
            cout << "Created new style: " << key << "\\n";
        }
        return styles[key];
    }
};

// Client
int main() {
    StyleFactory factory;

    // Characters share styles

```

```

auto style1 = factory.getStyle("Arial", 12, "Black");
style1->apply(0, 0, 'H');
style1->apply(1, 0, 'i');

auto style2 = factory.getStyle("Arial", 12, "Black");
style2->apply(2, 0, '!');

auto style3 = factory.getStyle("TimesNewRoman", 14, "Blue");
style3->apply(0, 1, 'T');
style3->apply(1, 1, 'e');
style3->apply(2, 1, 'x');
style3->apply(3, 1, 't');

return 0;
}

```

Proxy Design Pattern

Proxy is a structural design pattern that provides a placeholder or representative for another object to control how clients interact with it. It acts like a "middleman" between the client and the real object, adding an extra layer of control such as lazy loading, access checks, logging, or caching.

```

#include <iostream>

using namespace std;

// Subject interface

class Vehicle {
public:

```

```

    virtual void drive() = 0;
    virtual ~Vehicle() = default;
};

// Real subject
class Car : public Vehicle {
public:
    void drive() override {
        cout<< "Car is being driven"<<endl;
    }
};

// Proxy

class CarProxy : public Vehicle {
    Car* realCar;
    string driverLicense;

public:
    CarProxy(string license) : driverLicense(license), realCar(nullptr) {}

    void drive() override {
        if(driverLicense == "VALID") {
            if(nullptr == realCar) {
                realCar = new Car();
            }
            cout<< " Access granted"<<endl;
            realCar->drive();
        } else {
            cout<< "Access denied"<<endl;
        }
    }

    ~CarProxy() {
        delete realCar;
    }
};

int main() {
    Vehicle* car = new CarProxy("VALID");

```

```

car->drive();

Vehicle* car2 = new CarProxy("INVALID");
car2->drive();

return 0;
}

```

Chain of Responsibility Pattern

The Chain of Responsibility (CoR) pattern lets you pass requests along a chain of handlers. Each handler decides either:

- To process the request, or
- To pass it to the next handler in the chain.

This decouples senders from receivers and makes it easy to add/remove/change handlers.

```

#include <iostream>

using namespace std;

class SupportHandler {
    SupportHandler* nextHandler;
Public:
    SupportHandler() : nextHandler(nullptr) {}
    void setNextHandler(SupportHandler* next) {
        nextHandler = next;
    }

    virtual void handleRequest(const string& issue, int level) {
        if(nextHandler) {
            nextHandler->handleRequest(issue, level);
        } else {

```



```

        cout<<" No handler available"<<endl;
    }
}

virtual ~SupportHandler() {}
};

class BasicSupport : public SupportHandler {
Public:
    void handleRequest(const string& issue, int level) {
        if(level == 1) {
            cout<<"Level1 support resolved "<<issue<<endl;
        } else {
            SupportHandler::handleRequest(issue, level);
        }
    }
};

class IntermediateSupport : public SupportHandler {
Public:
    void handleRequest(const string& issue, int level) {
        if(level == 2) {
            cout<<"Level2 support resolved "<<issue<<endl;
        } else {
            SupportHandler::handleRequest(issue, level);
        }
    }
};

class Manager : public SupportHandler {
Public:
    void handleRequest(const string& issue, int level) {
        if(level == 3) {
            cout<<"Level3 support resolved "<<issue<<endl;
        } else {
            SupportHandler::handleRequest(issue, level);
        }
    }
};

```

```

    }
};

int main() {
    BasicSupport level1;
    IntermediateSupport level2;
    Manager manager;

    level1.setNextHandler(&level2);
    level2.setNextHandler(&manager);

    level1.handleRequest("Password reset", 1);
    level1.handleRequest("System crash", 2);

    return 0;
}

```

Command Pattern

The **Command pattern** encapsulates a request as an object, letting you:

- Parameterize clients with different requests
- Queue or log requests
- Support undo/redo operations

It turns a request into a command object with an `execute()` method. Can be used when you need to queue, log, or schedule requests or can pass a variety of family of requests for some operations

```

#include <iostream>
#include <stack>

using namespace std;

struct Command {
    virtual void execute() = 0;
    virtual void undo() = 0;
    virtual ~Command() {}
};

class TextEditor {
    string text;
Public:
    void write(const string& s) { text+= s; cout<<text<<endl; }
    void erase(int n) { text.erase(text.size() - n); cout<<text<<endl; }
    string get() { return text; }
};

// write Command
class WriteCommand : public Command {
    TextEditor* ed;
    string s;
Public:
    WriteCommand(TextEditor* e, string str) : ed(e), s(str) {}
    void execute() override { ed->write(s); }
    void undo() override { ed->erase(s.size()); }
};

// Erase command
class EraseCommand : public Command {
    TextEditor* ed;
    int n;
    string erased;
Public:
    EraseCommand(TextEditor* e, int n) : ed(e), n(n) {}
    void execute() override {

```

```

        string cur = ed->get();
        erased = cur.substr(cur.size() - n);
        ed->erase(n);
    }
    void undo() override { ed->write(erased); }
};

//Invoker
class EditorInvoker {
    stack<Command*> history;
Public:
    void run(Command* c) { c->execute(); history.push(c); }
    void undo() {
        if(!history.empty()) {
            history.top()->undo();
            history.pop();
        }
    }
};

int main() {
    TextEditor editor;
    EditorInvoker inv;

    inv.run(new WriteCommand(&editor, "Hello "));
    inv.run(new WriteCommand(&editor, "World!"));
    inv.run(new EraseCommand(&editor, 6));
    cout<<"Undoing ..."<<endl;
    inv.undo();
}

```

Mediator Design Pattern

Mediator is a behavioral design pattern that defines an object to encapsulate communication between multiple objects. Instead of objects talking to each other directly (causing tight coupling), they talk via a mediator. Promotes **loose coupling** and makes communication easier to manage.

Highly useful when you have **many-to-many communication** between objects.

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class User; // forward declaration

// Mediator
class ChatRoom {
public:
    void showMessage(User* user, const string& message);
};

// Colleague
class User {
    string name;
    ChatRoom* chatRoom;
public:
    User(string name, ChatRoom* room) : name(name), chatRoom(room) {}

    string getName() { return name; }

    void sendMessage(const string& msg) {
        chatRoom->showMessage(this, msg);
    }
};
```

```

// ConcreteMediator
void ChatRoom::showMessage(User* user, const string& message) {
    cout << user->getName() << " says: " << message << endl;
}

// Client
int main() {
    ChatRoom room;

    User user1("Alice", &room);
    User user2("Bob", &room);
    User user3("Charlie", &room);

    user1.sendMessage("Hello everyone!");
    user2.sendMessage("Hi Alice!");
    user3.sendMessage("Good morning guys!");

    return 0;
}

```

Memento Design Pattern

Memento is a behavioral design pattern that lets you save and restore an object's state without exposing its internal details. It is useful for implementing **undo/redo functionality**.

Components

1. **Originator** → The object whose state needs to be saved/restored.
2. **Memento** → Stores the state of the Originator (snapshot).
3. **Caretaker** → Manages the saved states (history), but never modifies them.

```

#include <iostream>
#include <string>
#include <stack>
using namespace std;

// Memento
class EditorMemento {
    string text;
public:
    EditorMemento(string text) : text(text) {}
    string getText() const { return text; }
};

// Originator
class TextEditor {
    string text;
public:
    void type(const string& words) {
        text += words;
        cout << "Current Text: " << text << endl;
    }

    EditorMemento save() { return EditorMemento(text); }

    void restore(const EditorMemento& memento) {
        text = memento.getText();
        cout << "Restored Text: " << text << endl;
    }
};

// Caretaker
class History {
    stack<EditorMemento> mementos;
public:
    void push(const EditorMemento& m) { mementos.push(m); }
    EditorMemento pop() {
        EditorMemento m = mementos.top();
        mementos.pop();
        return m;
    }
};

```

```
    }  
};  
  
// Client  
int main() {  
    TextEditor editor;  
    History history;  
  
    editor.type("Hello ");  
    history.push(editor.save());  
  
    editor.type("World!");  
    history.push(editor.save());  
  
    editor.type(" Extra text...");  
  
    // Undo operations  
    editor.restore(history.pop()); // Undo last  
    editor.restore(history.pop()); // Undo again  
  
    return 0;  
}
```

Observer Design Pattern

Observer is a behavioral design pattern where an object (subject) maintains a list of dependents (observers) and notifies them automatically of any state changes. This creates a one-to-many dependency between objects.

Purpose :

- When multiple objects need to stay in sync with another object.
- When you want to achieve **loose coupling** between publisher (subject) and subscribers (observers).
- Useful in **event-driven systems** (UI events, messaging systems, stock price updates).

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

// Observer
class Observer {
Public:
    virtual void update(float price) = 0;
    virtual ~Observer() = default;
};

// Subject
class Subject {
    vector<Observer*> observers;
Public:
    void addObserver(Observer* obs) { observers.push_back(obs); }
    void removeObserver(Observer* obs) {
        observers.erase(remove(observers.begin(), observers.end(), obs),
observers.end());
    }

    void notify(float price) {
        for (auto obs : observers) {
            obs->update(price);
        }
    }
};

// ConcreteSubject
class Stock : public Subject {
    float price;
public:
    void setPrice(float newPrice) {
        price = newPrice;
    }
};
```

```

        notify(price);
    }
};

// ConcreteObservers
class MobileApp : public Observer {
    string name;
Public:
    MobileApp(string n) : name(n) {}
    void update(float price) override {
        cout << "MobileApp " << name << " notified. New Stock Price: " << price
<< endl;
    }
};

class WebApp : public Observer {
    string name;
Public:
    WebApp(string n) : name(n) {}
    void update(float price) override {
        cout << "WebApp " << name << " notified. New Stock Price: " << price <<
endl;
    }
};

// Client
int main() {
    Stock googleStock;
    MobileApp m1("Alice");
    MobileApp m2("Bob");
    WebApp w1("Chrome");
    googleStock.addObserver(&m1);

    googleStock.addObserver(&m2);
    googleStock.addObserver(&w1);
    googleStock.setPrice(1200.5);
    googleStock.setPrice(1250.75);

    return 0;
}

```

Template Pattern

The Template Method pattern is a behavioral design pattern that defines the skeleton of an algorithm in a base class, but lets subclasses redefine certain steps of the algorithm without changing its overall structure.

```
#include <iostream>

using namespace std;

// Abstract

class DataParser {
Public:
    string filename;
    void ParseData(const string& filename) {
        this->filename = filename;
        openFile(filename);
        readData();
        parseContent();
        closeFile();
    }
    virtual ~DataParser() {}
Protected:
    void openFile(const string& filename) {
        cout<<" Opening File "<<filename << endl;
    }
    void readData() {
        cout<< " Reading data from file "<< filename <<endl;
    }

    void closeFile() {
        cout<<" Closing File"<<endl;
    }
    virtual void parseContent() = 0;
};

class JsonParser : public DataParser {
Protected:
```

```

        void parseContent() {
            cout<<" Parsing JSON data"<<endl;
        }
};

class XMLParser : public DataParser {
Public:
    void parseContent() {
        cout<<" Parsing XML data"<<endl;
    }
};

int main() {
    DataParser* jsonParser = new JsonParser();
    jsonParser->ParseData("Data.json");
    XMLParser* xmlParse = new XMLParser();
    xmlParse->ParseData("Data.xml");
    return 0;
}

```

Visitor Pattern

The Visitor pattern is a way to separate operations from the objects on which they operate. It lets you add new behaviors (operations) to an existing object structure without modifying those objects.

In other words:

- Objects (elements) just hold data and accept visitors.
- Visitors contain the logic (operations) you want to apply.

Think of it like a tourist (visitor) who visits different places (elements). Each place accepts the tourist and then lets the tourist perform activities there.

```

#include <iostream>
#include <vector>
using namespace std;

class File;
class Directory;

// Visitor interface
class Visitor {
public:
    virtual void visit(File& file) = 0;
    virtual void visit(Directory& dir) = 0;
    virtual ~Visitor() {}
};

// Element interface
class FileSystemElement {
public:
    virtual void accept(Visitor& v) = 0;
    virtual ~FileSystemElement() {}
};

// concrete elements
class File : public FileSystemElement {
    string name;
    int size;
public:
    File(string n, int s) : name(n), size(s) {}
    string getName() const {
        return name;
    }

    int getSize() const {
        return size;
    }

    void accept(Visitor& v) override {
        v.visit(*this);
    }
};

```

```

class Directory : public FileSystemElement {
    string name;
    vector<FileSystemElement*> children;
Public:
    Directory(string n) : name(n) {}
    string getName() const {
        return name;
    }

    void add(FileSystemElement* elem) {
        children.push_back(elem);
    }

    vector<FileSystemElement*> getChildren() const {
        return children;
    }

    void accept(Visitor& v) {
        v.visit(*this);
    }

    ~Directory() {
        for(auto child : children) {
            delete child;
        }
    }
};

```

```

class SizeCalculator : public Visitor {
    int totalsize = 0;
Public:
    void visit(File& file) override {
        totalsize += file.getSize();
    }

    void visit(Directory& dir) override {
        for(auto child : dir.getChildren()) {
            child->accept(*this);
        }
    }
}

```

```

    }

    int getTotalSize() const {
        return totalsize;
    }
};

int main() {
    Directory* root = new Directory("root");
    root->add(new File("file1.txt", 10));
    root->add(new File("file2.txt", 10));

    Directory* subDir = new Directory("docs");
    subDir->add(new File("resume.txt", 20));
    root->add(subDir);

    SizeCalculator sizeVisitor;
    root->accept(sizeVisitor);
    cout<< " Total size : " << sizeVisitor.getTotalSize() <<" KB"<<endl;

    return 0;
}

```

Strategy Design Pattern

Strategy is a behavioral design pattern that lets you define a family of algorithms, put each of them in a separate class, and make their objects interchangeable at runtime.

The key idea is that instead of hardcoding an algorithm inside a class, we encapsulate it in different strategy classes and let the client choose which one to use.

```
#include <iostream>
#include <memory>

using namespace std;

// interface for strategy
class PaymentStrategy {
public:
    virtual void pay(int amount) = 0;
    virtual ~PaymentStrategy() {}
};

// concrete strategy
class CreditCardPayment : public PaymentStrategy {
public:
    void pay(int amount) override {
        cout<< "Paid "<< amount << " using credit card"<<endl;
    }
};

class PhonePe : public PaymentStrategy {
public:
    void pay(int amount) override {
        cout<< "Paid "<< amount << " using phonepe"<<endl;
    }
};

// context
class ShoppingCart {
    unique_ptr<PaymentStrategy> strategy;
public:
```



```

void setPaymentStrategy(PaymentStrategy* s) {
    strategy.reset(s);
}

void checkout(int amount) {
    if(strategy) strategy->pay(amount);
    else cout<< "Invalid payment method"<<endl;
}
};

int main() {
    ShoppingCart cart;

    cart.setPaymentStrategy(new CreditCardPayment());
    cart.checkout(100);

    cart.setPaymentStrategy(new PhonePe());
    cart.checkout(200);

    return 0;
}

```

State Design Pattern

The State Pattern allows an object to alter its behavior when its internal state changes. It appears as if the object changed its class.

Instead of using a long chain of if/else or switch, state-specific logic is put into separate state classes.

```

#include <iostream>

using namespace std;

class MediaPlayer;

// State interface
class PlayerState {
public:
    virtual void play(MediaPlayer* player) = 0;
    virtual void pause(MediaPlayer* player) = 0;
    virtual void stop(MediaPlayer* player) = 0;
    virtual ~PlayerState() {}
};

// context
class MediaPlayer {
    PlayerState* state;
public:
    MediaPlayer(PlayerState* initialState) : state(initialState) {}
    void setState(PlayerState* newState) {
        state = newState;
    }

    void play() {
        state->play(this);
    }
    void pause() {
        state->pause(this);
    }
    void stop() {
        state->stop(this);
    }
};

// Concrete States
class PlayingState : public PlayerState {
public:

```

```

    void play(MediaPlayer* player) override {
        cout<<"Already playing"<<endl;
    }
    void pause(MediaPlayer* player) override;
    void stop(MediaPlayer* player) override;
};

class PausedState : public PlayerState {
public:
    void play(MediaPlayer* player) override;
    void pause(MediaPlayer* player) override {
        cout<<"Already paused"<<endl;
    }
    void stop(MediaPlayer* player) override;
};

class StoppedState : public PlayerState {
public:
    void play(MediaPlayer* palyer) override;
    void pause(MediaPlayer* palyer) override {
        cout<<"Can't pause, player is stopped"<<endl;
    }
    void stop(MediaPlayer* player) override {
        cout<<"Already stopped"<<endl;
    }
};

// state transitions
void PlayingState::pause(MediaPlayer* player) {
    cout<<"Pausing playback"<<endl;
    static PausedState paused;
    player->setState(&paused);
}

void PlayingState::stop(MediaPlayer* player) {
    cout<<"Stopping playback"<<endl;
    static StoppedState stopped;
    player->setState(&stopped);
}

```

```

void PausedState::play(MediaPlayer* player) {
    cout<<"Resuming playback"<<endl;
    static PlayingState playing;
    player->setState(&playing);
}

void PausedState::stop(MediaPlayer* player) {
    cout<<"Stopping from pause"<<endl;
    static StoppedState stopped;
    player->setState(&stopped);
}

void StoppedState::play(MediaPlayer* player) {
    cout<<"Starting playback"<<endl;
    static PlayingState playing;
    player->setState(&playing);
}

// client
int main() {
    static StoppedState stopped;
    MediaPlayer player(&stopped);
    player.play();
    player.pause();
    player.play();
}

```

*** END ***